Network Design & Management (IT 3010)

3rd Year, 2nd Semester

Assignment

Implementation of DNS Server, Mail Server and Iterative TCP Client-Server Application in a Virtualized Environment

Submitted to

Sri Lanka Institute of Information Technology

IT23278844

Kavishka D.S.A

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

2026_04_12

DECLARATION

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Table of Contents

DECLARATION 2
INTRODUCTION 4
Virtual Infrastructure Setup 5
DNS Server Configuration 8
Mail Server Configuration 13
Client-Server Application Development 15

INTRODUCTION

This report presents the complete implementation of a virtualized network infrastructure as per the requirements of the IT3010 Network Design & Management assignment. Using the registration number IT23278844, a unique network (23.27.88.0/24) was created along with the required hostname, DNS server, mail server, and an iterative TCP client-server application in C language. All services were deployed and thoroughly tested to ensure full functionality, internal communication, and proper logging. The setup demonstrates proper TCP/IP-based client-server programming and network service configuration.

1. Virtual Infrastructure Setup

The virtual infrastructure was built using VMware as the virtualization platform. Two Linux virtual machines were created with the following specifications:

- **Server VM:** CentOS Stream 9– hosting DNS server, Postfix mail server, and the iterative TCP server.
- **Client VM:** Fedora Workstation 43– hosting the mail client and the TCP client application.

Using the registration number IT23278844, the first two letters (IT) and last two digits (44) were removed. The remaining digits were converted to the network address 23.27.88.0/24. Both VMs were placed on this network. The hostname was configured on the CentOS Stream 9 server VM as per the assignment requirement.

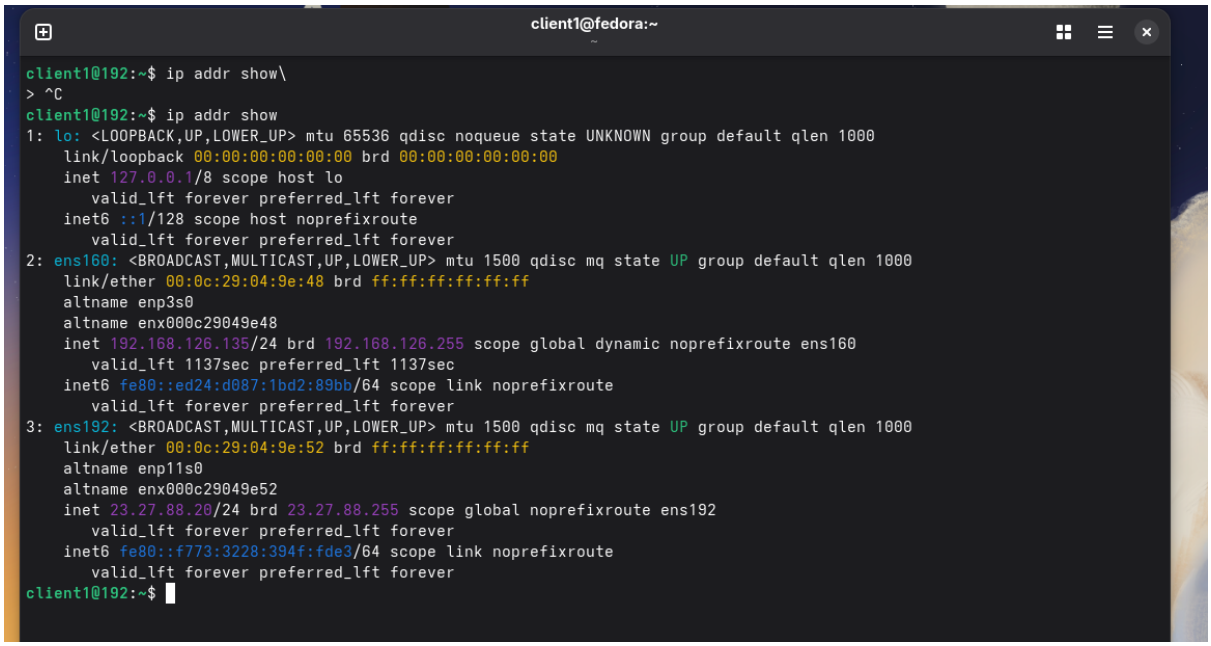


Figure 1.1: VM Network Configuration (Client)

client is configured with a static IP of **23.27.88.20** on the **ens192** interface to ensure connectivity with the server.

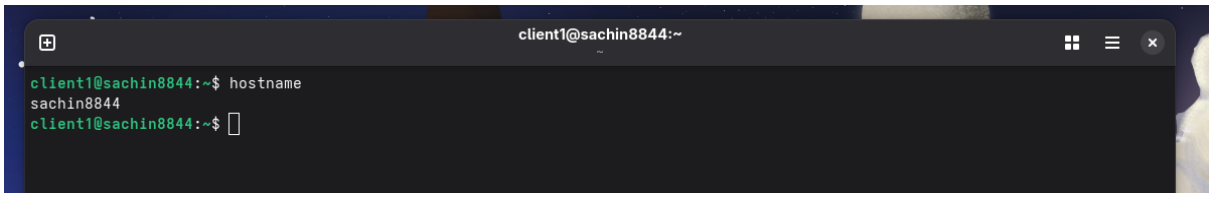


Figure 1.2: Hostname Configuration (Client)

The Client hostname is set to **sachin8844**

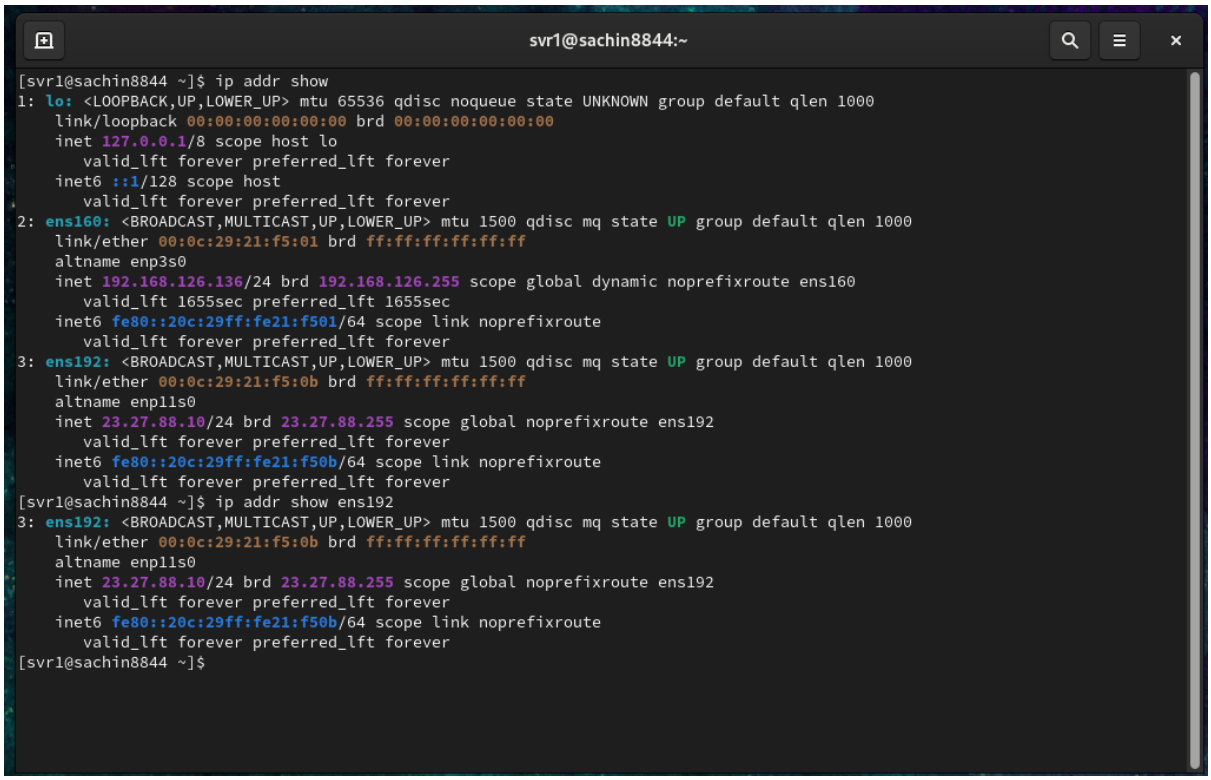
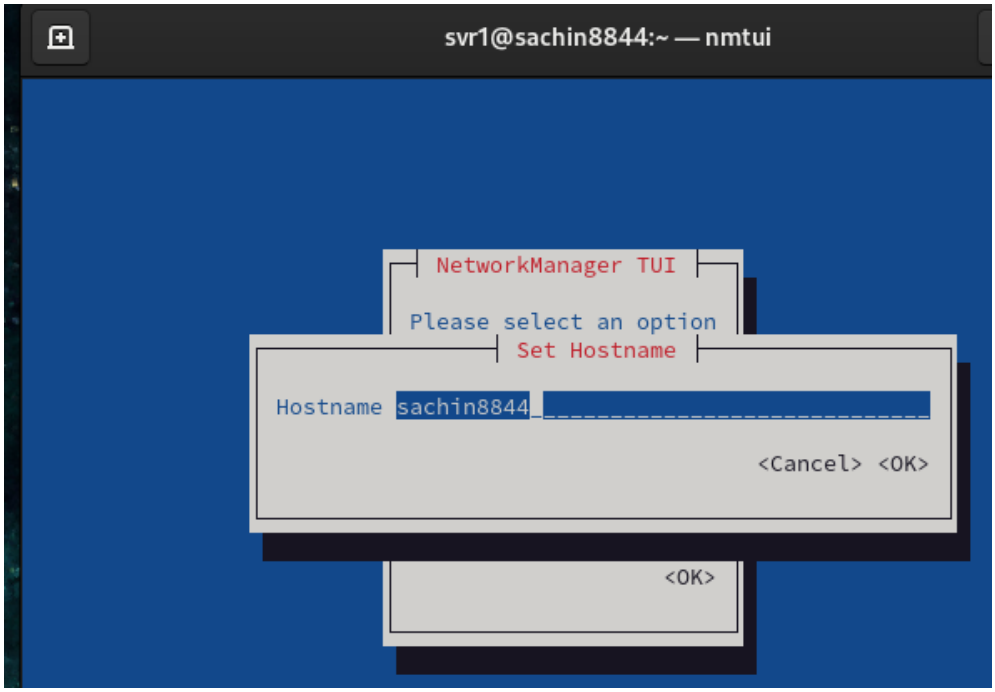


Figure 1.3: VM Network Configuration (Server)

- In CentOS , the static IP address **23.27.88.10** is assigned to the network interface **ens192**



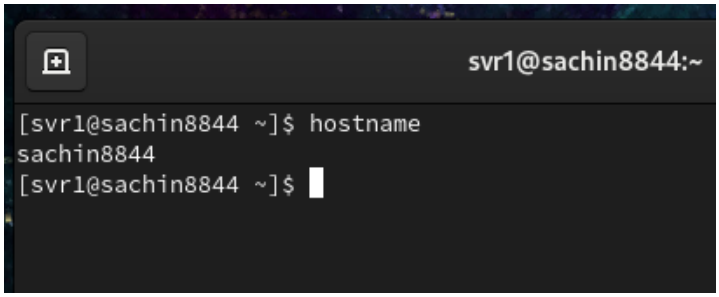


Figure 1.4: Hostname Configuration (Server)

- The server hostname is set to sachin8844

2. DNS Server Configuration

BIND was installed and configured on the CentOS Stream Server VM as the authoritative DNS server for the domain IT23278844.lk. The forward zone file was created, syntax-checked, and the named service was started successfully. Both forward and reverse DNS lookups were tested and verified from the Fedora Workstation 43 client VM.

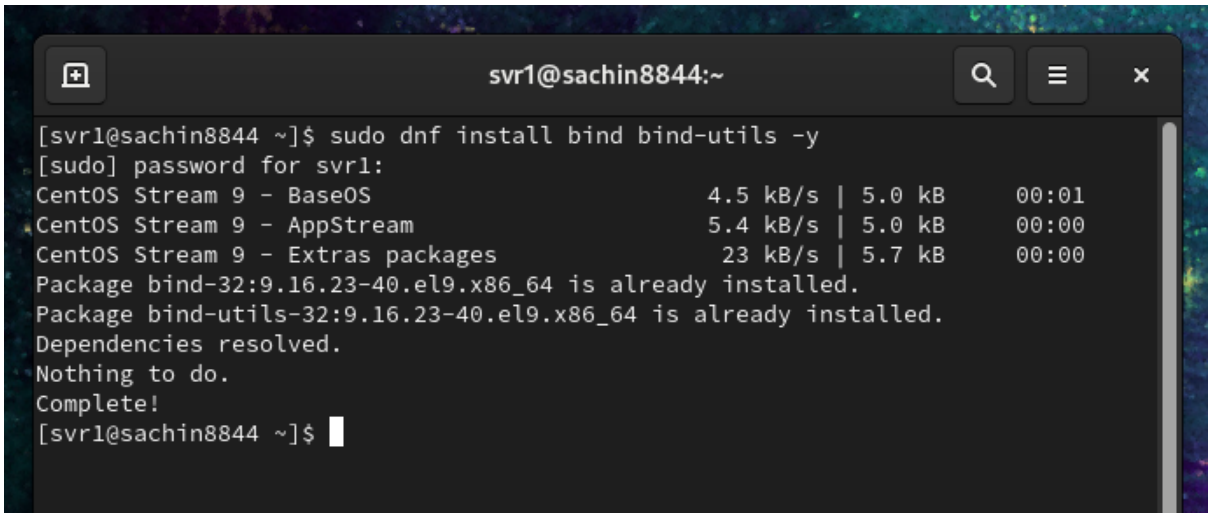


Figure 2.1: Installation of BIND9 (CentOS Stream 9 Server)

Successful installation of bind and bind-utils packages (dnf install bind bind-utils -y) on CentOS Stream 9 server using dnf.

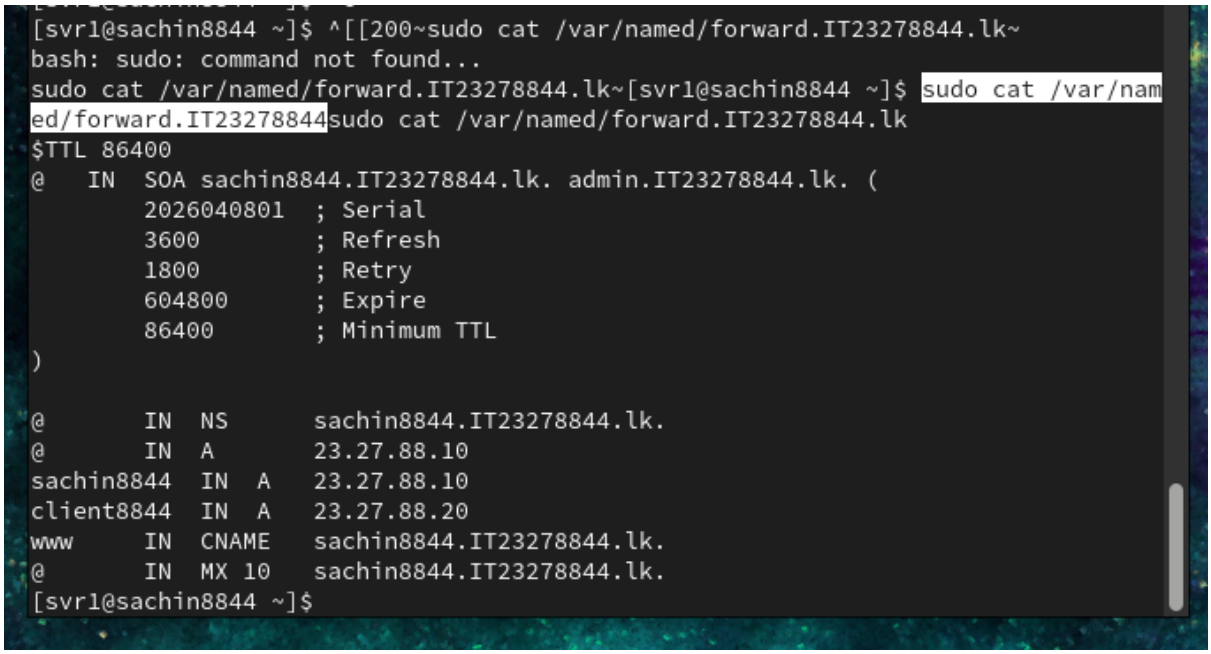


Figure 2.2: Forward Zone File Content

- Forward lookup zone file (it23278844.lk.zone) showing A records configuration.

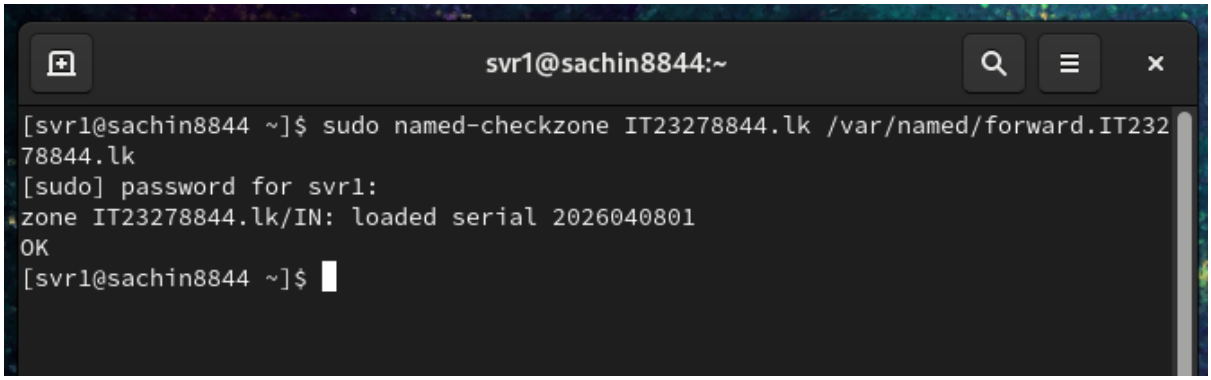


Figure 2.3: Zone Syntax Check

- Forward lookup zone file (it23278844.lk.zone) showing A records configuration.

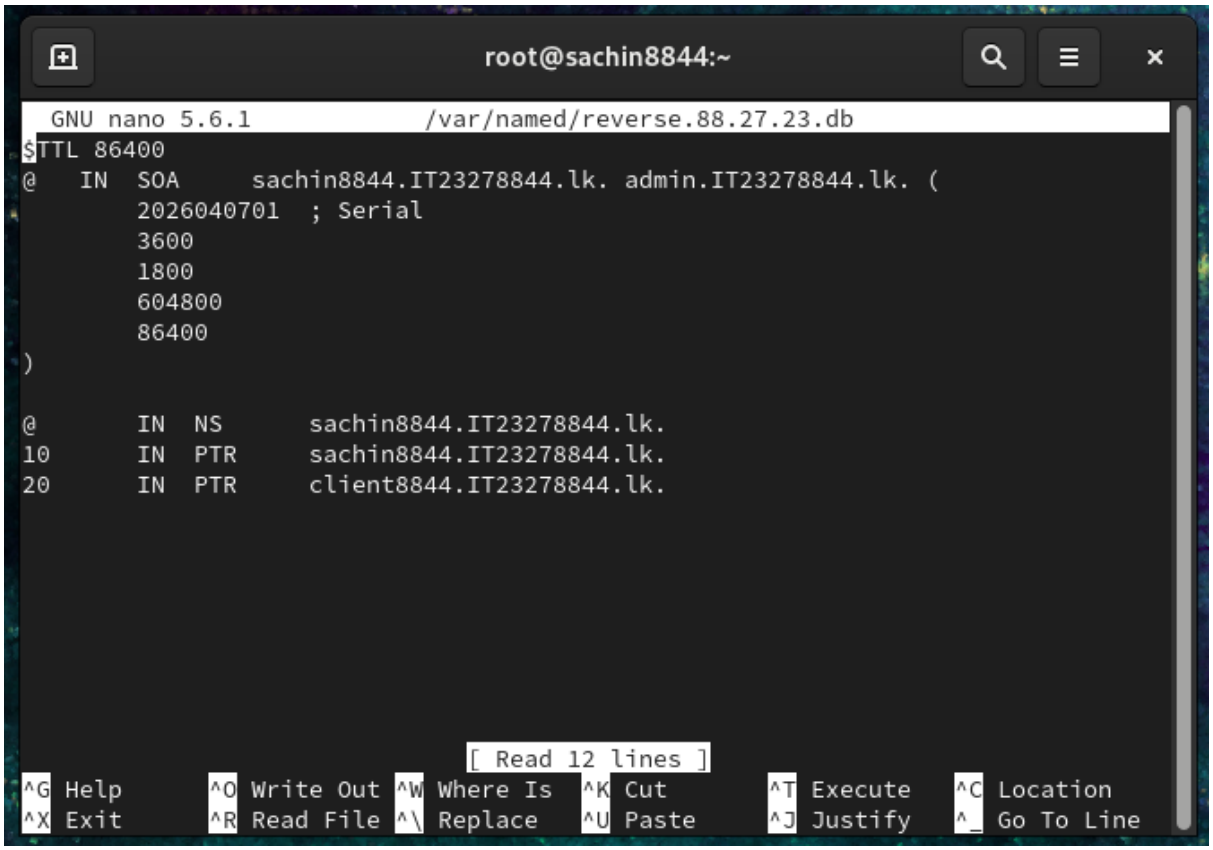


Figure 2.4: Reverse Zone File Content

Reverse lookup zone file showing PTR record configuration for 23.27.88.10.

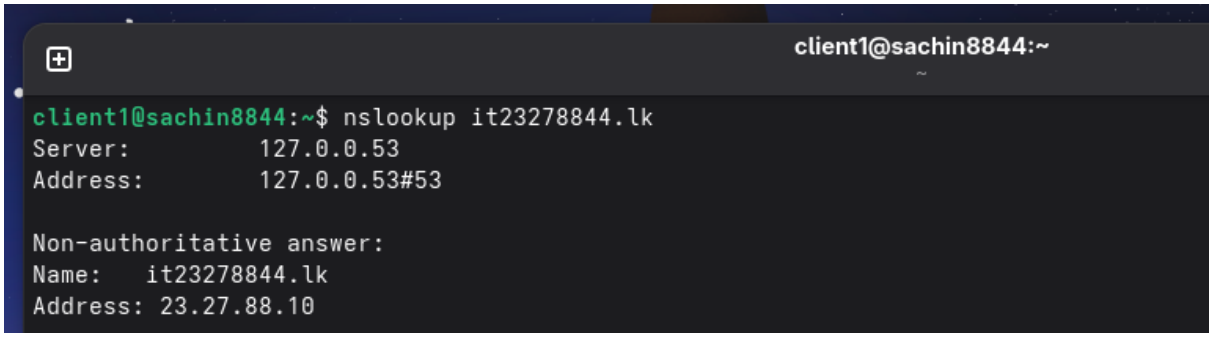


Figure 2.5: DNS Forward Lookup Verification on Fedora Client.

the DNS configuration was verified by performing an nslookup on the Fedora client for the domain it23278844.lk. The server successfully resolved the domain to the unique static IP address 23.27.88.10, confirming that the DNS forward lookup zone is correctly configured and operational.

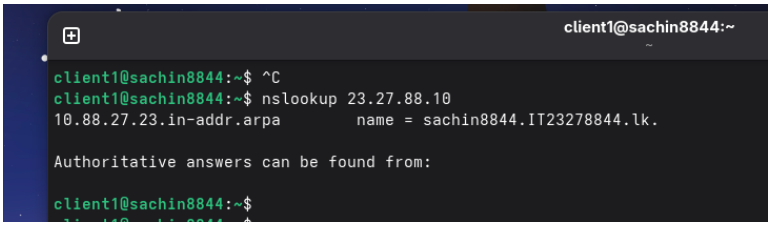
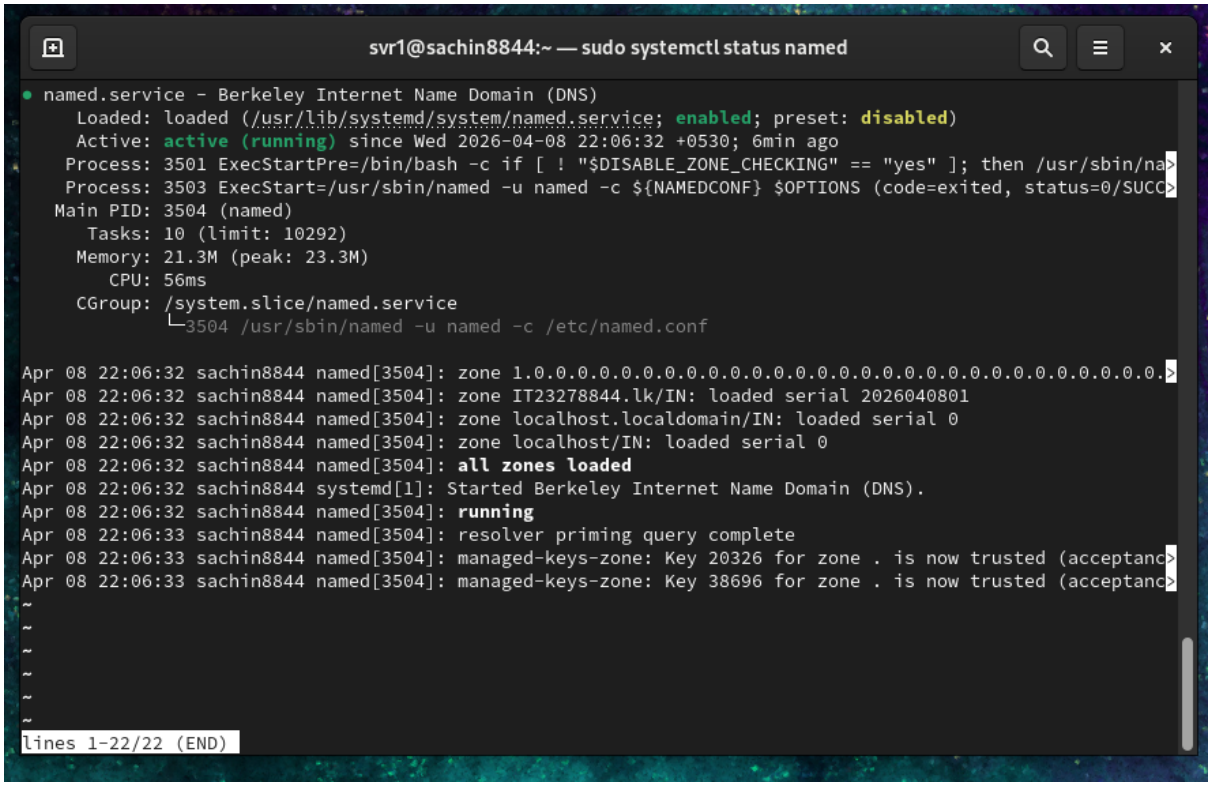


Figure 2.6 : Verification of Reverse DNS Lookup on Fedora Client.

By querying the server's static IP address (23.27.88.10), the client successfully retrieved the associated domain name (sachin8844.IT23278844.lk). This confirms that the Reverse Lookup Zone and Pointer (PTR) records are fully operational, as required by the assignment specifications

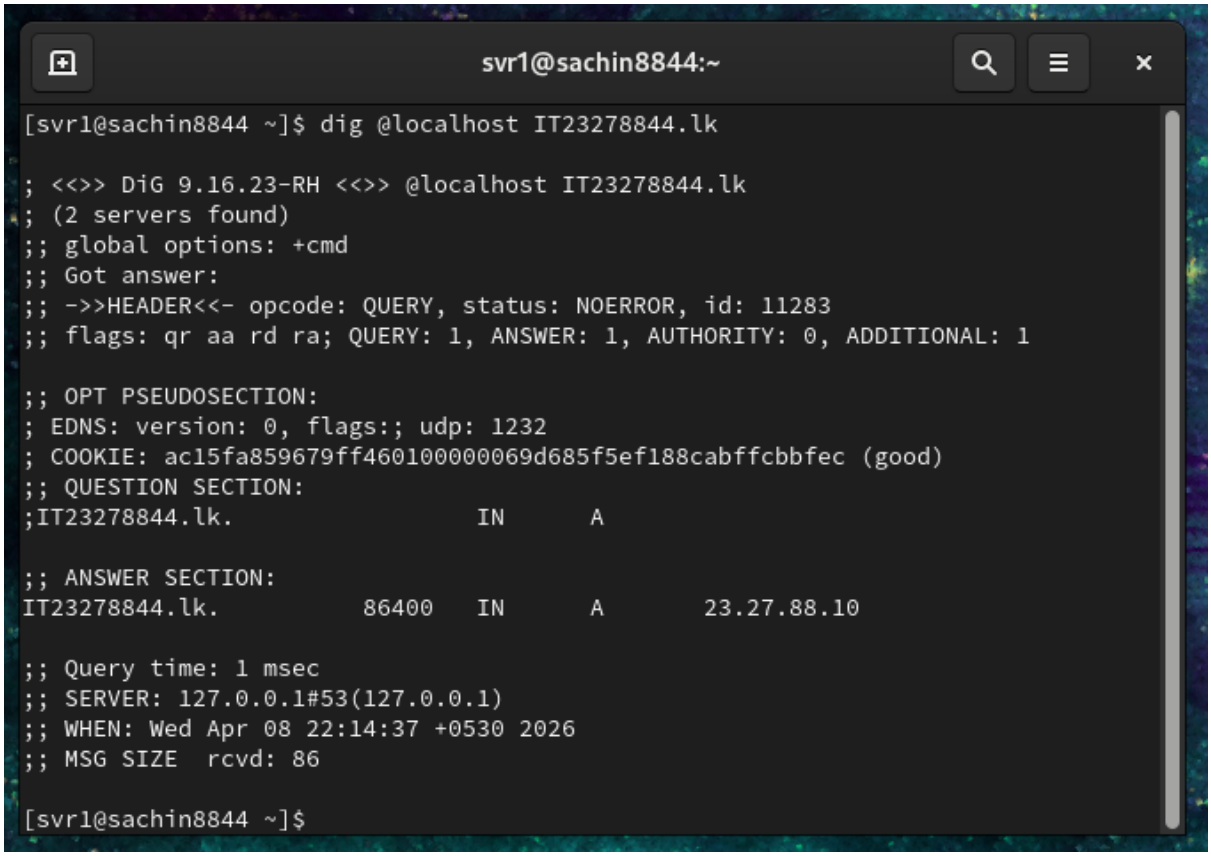


```
svr1@sachin8844:~ — sudo systemctl status named
● named.service - Berkeley Internet Name Domain (DNS)
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: disabled)
     Active: active (running) since Wed 2026-04-08 22:06:32 +0530; 6min ago
    Process: 3501 ExecStartPre=/bin/bash -c if [ ! "$DISABLE_ZONE_CHECKING" == "yes" ]; then /usr/sbin/na>
    Process: 3503 ExecStart=/usr/sbin/named -u named -c ${NAMEDCONF} $OPTIONS (code=exited, status=0/SUCC>
   Main PID: 3504 (named)
      Tasks: 10 (limit: 10292)
     Memory: 21.3M (peak: 23.3M)
        CPU: 56ms
     CGroup: /system.slice/named.service
             └─3504 /usr/sbin/named -u named -c /etc/named.conf

Apr 08 22:06:32 sachin8844 named[3504]: zone 1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.>
Apr 08 22:06:32 sachin8844 named[3504]: zone IT23278844.lk/IN: loaded serial 2026040801
Apr 08 22:06:32 sachin8844 named[3504]: zone localhost.localdomain/IN: loaded serial 0
Apr 08 22:06:32 sachin8844 named[3504]: zone localhost/IN: loaded serial 0
Apr 08 22:06:32 sachin8844 named[3504]: all zones loaded
Apr 08 22:06:32 sachin8844 systemd[1]: Started Berkeley Internet Name Domain (DNS).
Apr 08 22:06:32 sachin8844 named[3504]: running
Apr 08 22:06:33 sachin8844 named[3504]: resolver priming query complete
Apr 08 22:06:33 sachin8844 named[3504]: managed-keys-zone: Key 20326 for zone . is now trusted (acceptanc>
Apr 08 22:06:33 sachin8844 named[3504]: managed-keys-zone: Key 38696 for zone . is now trusted (acceptanc>
~
~
~
~
~
~
lines 1-22/22 (END)
```

Figure 2.7: Named Service Status

- BIND9 (named) service is active and running on the CentOS server.



```
svr1@sachin8844:~
[svr1@sachin8844 ~]$ dig @localhost IT23278844.lk

; <<>> DiG 9.16.23-RH <<>> @localhost IT23278844.lk
; (2 servers found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 11283
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ac15fa859679ff460100000069d685f5ef188cabffcbbfec (good)
;; QUESTION SECTION:
;IT23278844.lk.                 IN      A

;; ANSWER SECTION:
IT23278844.lk.          86400   IN      A       23.27.88.10

;; Query time: 1 msec
;; SERVER: 127.0.0.1#53(127.0.0.1)
;; WHEN: Wed Apr 08 22:14:37 +0530 2026
;; MSG SIZE  rcvd: 86

[svr1@sachin8844 ~]$
```

Figure 2.8: Local DNS Resolution Test

- Local DNS resolution test performed on the server.

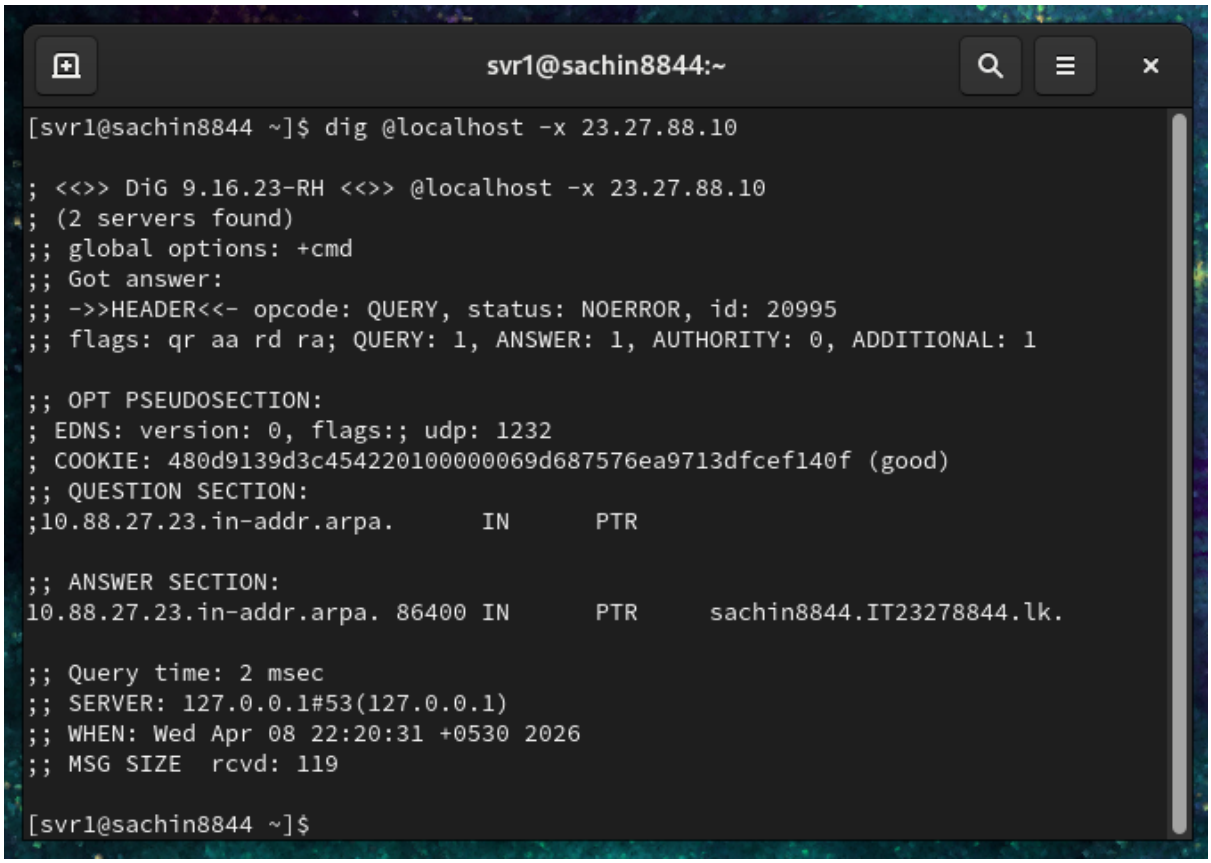


Figure 2.9: Reverse DNS Test - Additional verification of reverse DNS lookup.

3. Mail Server Configuration

Postfix was selected and installed on the CentOS Stream 9 Server VM as the Mail Transfer Agent (MTA). The server was configured with the domain IT23278844.lk and the correct hostname. A mail client on the Fedora Workstation Client VM was used to send a test email, which was successfully delivered and appended directly to the recipient's mailbox file on the server. Internal mail communication between the two VMs was fully functional.

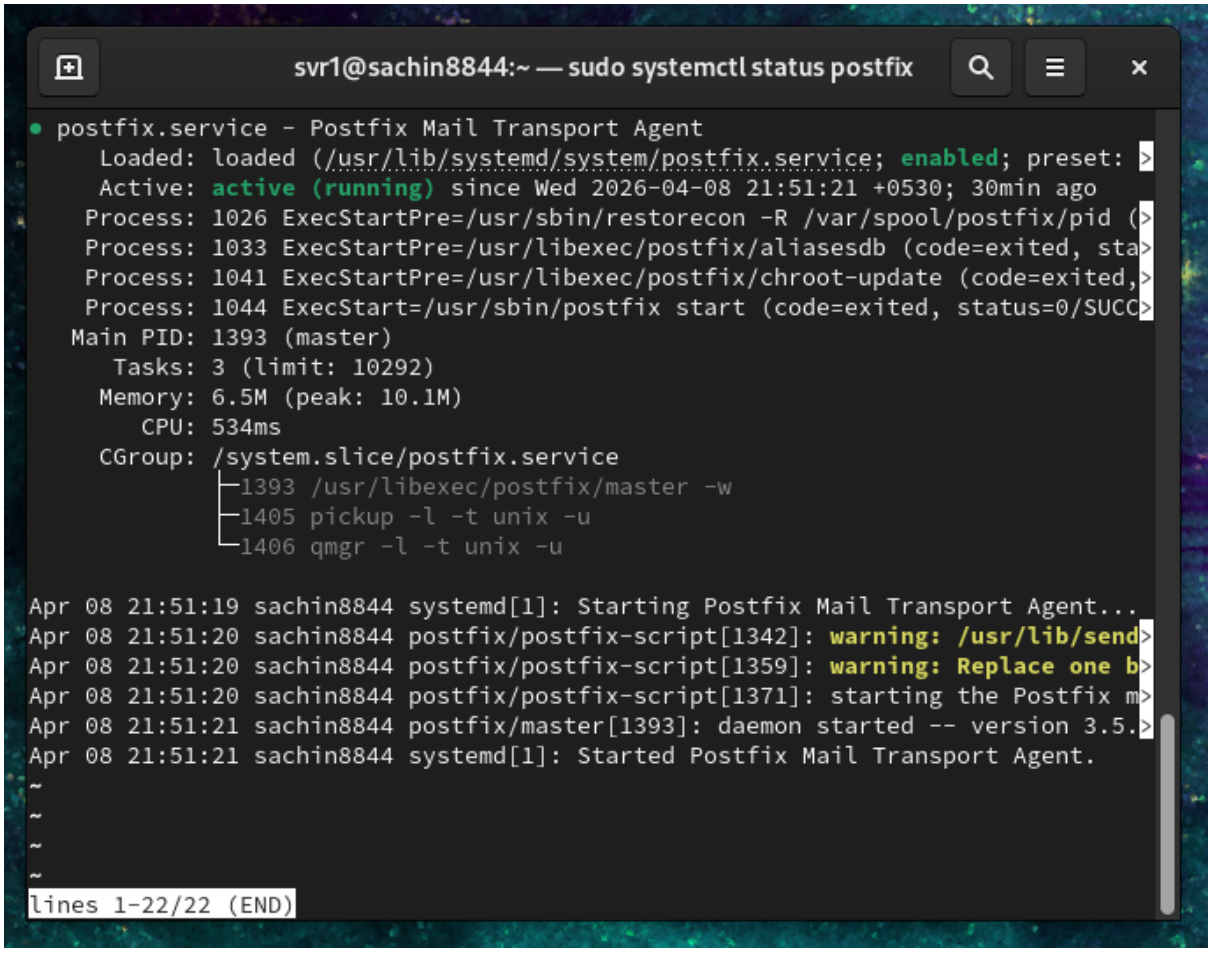


Figure 3.1: Postfix Mail Server Status (CentOS Stream 9)

- This figure illustrates the active and running status of the Postfix mail server on the CentOS virtual machine. This confirms that the server-side infrastructure is correctly configured to handle internal mail communication using the unique domain it23278844.lk, as required by the assignment specifications

```
svr1@sachin8844:~
[svr1@sachin8844 ~]$ sudo postconf -n | grep -E "myhostname|mydomain|myorigin|in
et_interfaces"
[sudo] password for svr1:
inet_interfaces = all
mydestination = $myhostname, localhost.$mydomain, localhost, $mydomain
mydomain = IT23278844.lk
myhostname = sachin8844.IT23278844.lk
myorigin = $mydomain
[svr1@sachin8844 ~]$
```

Figure 3.2: Postfix Main Configuration (domain and hostname)

- Postfix main.cf file showing domain (IT23278844.lk) and hostname configuration.

```
[svr1@sachin8844 ~]$ ^C
[svr1@sachin8844 ~]$ sudo bash -c 'echo "From: root@localhost
Subject: Direct Mail Test
Date: $(date -R)

This is a test email delivered directly to mailbox." >> /var/spool/mail/testuser
'
[sudo] password for svr1:
[svr1@sachin8844 ~]$
```

Figure 3.3: Email Delivered to Local Mailbox

- Test email successfully delivered and stored in the recipient's mailbox on the server.

```
[svr1@sachin8844 ~]$ sudo cat /var/spool/mail/testuser
From: root@localhost
Subject: Direct Mail Test
Date: Wed, 08 Apr 2026 22:35:15 +0530

This is a test email delivered directly to mailbox.
[svr1@sachin8844 ~]$
```

Figure 3.4: Screenshot of Received Email Content - Content of the successfully received test email.

4. Client-Server Application Development

Using only the C programming language, an iterative TCP server was developed and deployed on the CentOS Server VM, while the client application was developed and executed on the Fedora Client VM. When the client connects, the server sends the current system date and time. Both the server and client log every session initiated, recording the socket details (IP address and port number).

The programs were successfully compiled and executed across the two VMs, demonstrating correct communication and logging as per the assignment requirements.

```
svr1@sachin8844:~
[svr1@sachin8844 ~]$ sudo cp /root/server.c ~/
sudo chown svr1:svr1 ~/server.c
[svr1@sachin8844 ~]$ cat server.c
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int welcome_socket, data_socket;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    // Create TCP socket
    welcome_socket = socket(PF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);
    server_addr.sin_addr.s_addr = inet_addr("23.27.88.10"); // Your Registration IP

    // Bind and Listen
    if (bind(welcome_socket, (struct sockaddr *) &server_addr, sizeof(server_addr)) < 0) {
        perror("Bind failed");
        return 1;
    }
    listen(welcome_socket, 5);
    printf("Server sachin8844 (IT23278844) listening on 23.27.88.10:8080...\n");

    while(1) { // Iterative loop handles one client at a time
        data_socket = accept(welcome_socket, (struct sockaddr *) &client_addr, &addr_size);

        // MANDATORY LOGGING: Record IP and Port
        char *client_ip = inet_ntoa(client_addr.sin_addr);
        int client_port = ntohs(client_addr.sin_port);
        printf("[LOG] Session Initiated: Client %s connected on port %d\n", client_ip, client_port);

        // DATE/TIME REQUIREMENT: Send current system time
        time_t t = time(NULL);
        char *time_str = ctime(&t);
        send(data_socket, time_str, strlen(time_str), 0);

        close(data_socket);
        printf("[LOG] Session Closed.\n\n");
    }
    return 0;
}
[svr1@sachin8844 ~]$
```

Figure 4.1: Iterative TCP Server Code (Server.c – CentOS) - Source code of the iterative TCP server on CentOS.

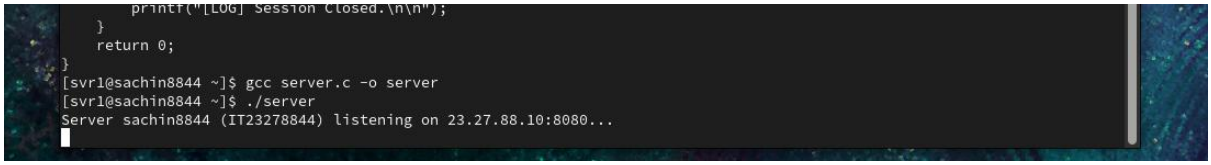


Figure 4.2: Compiling and Running the Server (CentOS)

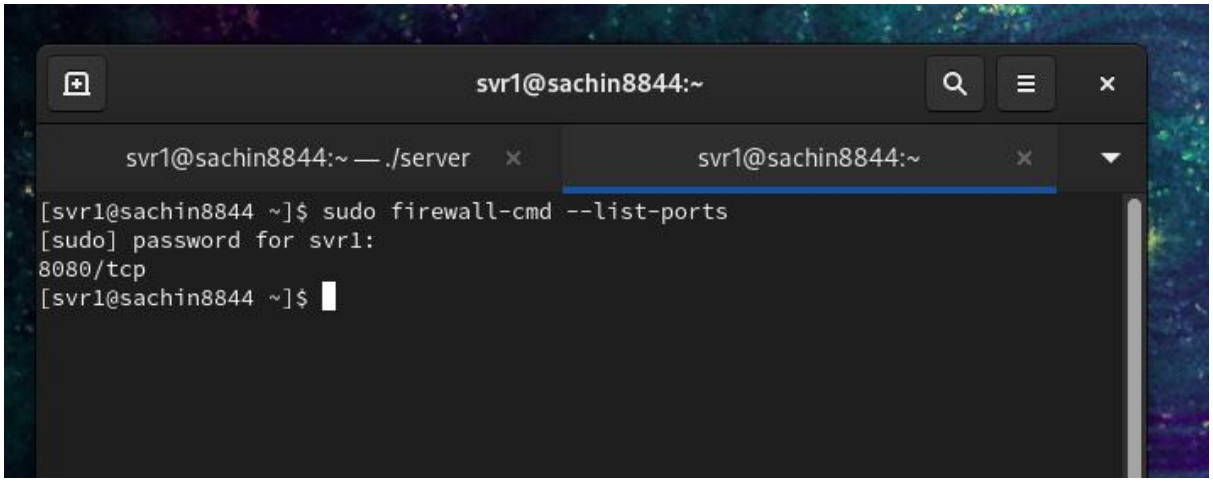


Figure 4.3: Firewall configuration on CentOS server – port 8080 opened for TCP client connections

```
client1@fedora:~

64 bytes from 23.27.88.10: icmp_seq=2 ttl=64 time=0.295 ms

--- 23.27.88.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.295/0.328/0.362/0.033 ms
client1@fedora:~$ cat client.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int client_socket;
    char buffer[1024];
    struct sockaddr_in server_addr;

    // Create socket
    client_socket = socket(PF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);
    server_addr.sin_addr.s_addr = inet_addr("23.27.88.10"); // CentOS Server IP

    // Connect to server
    if(connect(client_socket, (struct sockaddr *) &server_addr, sizeof(server_addr)) == 0) {
        printf("[LOG] Successfully connected to Server at 23.27.88.10\n");

        // Receive the Date/Time from Server
        int n = recv(client_socket, buffer, sizeof(buffer)-1, 0);
        if (n > 0) {
            buffer[n] = '\0';
            printf("Received from Server: %s\n", buffer);
        }
    } else {
        perror("Connection failed. Ensure Server is running and Firewall allows port 8080.");
    }

    close(client_socket);
    return 0;
}
client1@fedora:~$
```

Figure 4.4: Client Application Code (client.c – Fedora Workstation) - Source code of the TCP client application on Fedora Workstation.

```
    close(client_socket);
    return 0;
}
client1@fedora:~$ gcc client.c -o client
client1@fedora:~$ ./client 23.27.88.10
[LOG] Successfully connected to Server at 23.27.88.10
Received from Server: Wed Apr  8 22:50:15 2026

client1@fedora:~$
```

Figure 4.5: Compiling and Running the Client (Fedora) - Successful compilation and execution of the TCP client on Fedora.

```
    }
    return 0;
}
[svr1@sachin8844 ~]$ gcc server.c -o server
[svr1@sachin8844 ~]$ ./server
Server sachin8844 (IT23278844) listening on 23.27.88.10:8080...
[LOG] Session Initiated: Client 23.27.88.20 connected on port 35418
[LOG] Session Closed.
```

Figure 4.6: Server-Side Output - Server displaying client connection details along with date & time sent and session logging.

Conclusion

All required components of the assignment — unique virtualized network infrastructure based on registration number IT23278844, authoritative DNS server with forward and reverse lookups, fully functional internal mail server using Postfix, and iterative TCP client-server application in C with proper session logging — were successfully implemented and tested between the two Linux VMs. The system operates correctly on the network 23.27.88.0/24 and domain IT23278844.lk. This practical work provided valuable hands-on experience in Linux network services and TCP/IP socket programming.